Architectures Logicielles et Matérielles
Cours 02 – Architecture d'un microcontrôleur

Bruno Ferres
basé sur un cours de Philippe Waille

10 septembre 2026

Contenu de la séance

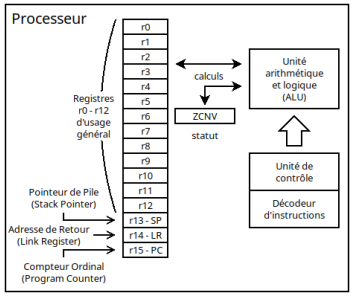
Rappels de L2 (INF401)

- architecture Von Neumann :
  un CPU, une mémoire, des bus et des périphériques
- représentation des données en machine

Nouveautés : gestion des périphériques, microcontrôleurs

- détails sur les accès à la mémoire
- configuration et communication avec des périphériques d'entrée/sortie
- découverte de la carte cible *Discovery*

Composant principal : le processeur

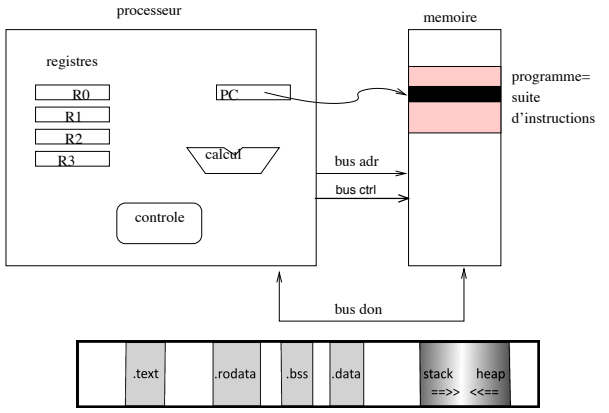


Composants d'un processeur

- calcul : une Unité Arithmétique et Logique (UAL)
- mémorisation : des "petites mémoires" – les registres
- contrôle : une Unité de Contrôle
  ↪ pour l'interprétation des instructions (voir cours 05)

crédit : équipe pédagogique IESE Polytech Grenoble

L'héritage de Von Neumann : vision globale



Caractéristiques

- une mémoire **unique** – données (variables) & programme (instructions)
- exécution **séquentielle** des instructions dans l'ordre de la mémoire

Contenu du processeur : la mémoire

Rappels : octets et mots

- un octet = 8 bits
- un mot = unité de base en mémoire, **pour une archi. donnée**
  - ↪ taille des adresses / taille maximale d'un accès mémoire
  - ↪ taille des registres et des calculs
  - ↪ taille des bus de données et d'adresse (voir la suite du cours)
- un mot = 32 ou 64 bits (*actuellement*)

Contenu de la mémoire

- la mémoire = un tableau d'octets, groupés en **mots**
- contient les **instructions** (le programme) et les **données** (variables)
- une adresse = le numéro (l'indice) d'un octet dans la mémoire
  - ↪ plus petite granularité des accès à la mémoire

Granularité des accès mémoires

Une histoire d'adresse

La poste a une adresse pour votre domicile...
... mais pas pour votre chambre, dans votre domicile.

Le processeur a une adresse pour chaque octet en mémoire...
... **mais pas pour chaque bit dans chaque octet !**

Comment accéder à un bit en particulier ?

1. charger l'octet contenant le bit depuis la mémoire (si lecture)
2. opérations logiques (`|`, `&`, `^`) sur l'octet
3. stocker l'octet modifié en mémoire (si écriture)

Contenu du processeur : les registres comme opérande

Rôle des registres

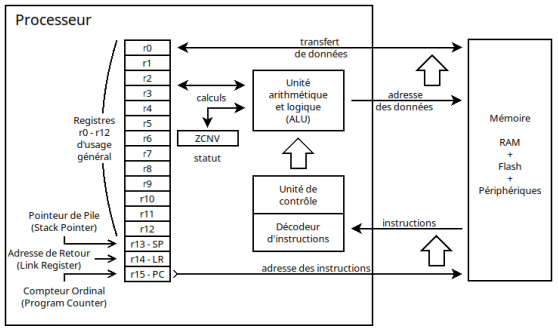
- un registre = une petite mémoire d'un mot
- les registres = opérandes des instructions
  - ↪ `add r0, r1, r2` sémantique : *r0* ← *r1* + *r2*
- un registre particulier : `pc`
  - ↪ contient l'adresse de **l'instruction courante**
  - ↪ `pc += 4` après chaque instruction (exécution **séquentielle**)

Registres et accès mémoire

Les registres peuvent contenir des adresses (*e.g.* `pc`, mais pas que)

- `ldr r1, [r2]` *r1* ← *Mem[r2]*, où `r2` contient une adresse
- `str r1, [r2]` *Mem[r2]* ← *r1*, où `r2` contient une adresse

L'héritage de Von Neumann : quid des périphériques ?



- on accède aux périphériques via les mêmes bus
- périphériques vus comme des **mots mémoire** par le CPU
  - ↪ un périphérique = une ou plusieurs adresses dédiées

crédit : équipe pédagogique IESE Polytech Grenoble

Synoptique d'un ordinateur

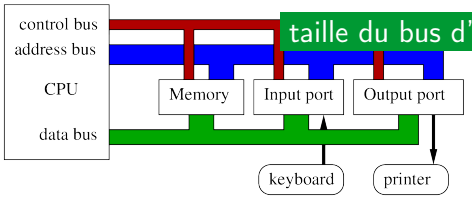
Composants d'un ordinateur

1. le processeur (CPU) : registres et calcul
2. la mémoire (RAM) : données et programme
3. les entrées/sorties (I/O) : environnement

Interconnexion via des bus

Un bus = un ensemble (#**fixe**) de fils

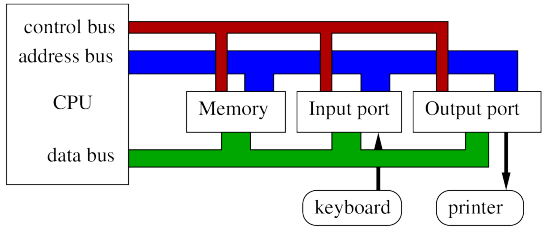
- données : contenu lu ou écrit 1 (b), 2 (h) ou 4 (w) mots
- adresse : numéro des octets accédés
- contrôle : moment + sens (R/W) + taille du transfert



taille du bus d'adresse en `ARM` ?

Microcontrôleur : ordinateur monopuce (CPU + RAM + I/O)

Signaux du bus de contrôle



Quand et dans quel "sens" accède-t-on à la mémoire ?

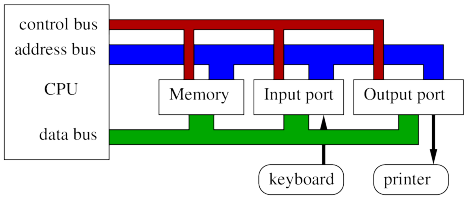
Une paire de signaux : (`Access`, $r/\overline{w}$) ou (`rd`,`wr`).

`Access`	$r/\overline{w}$	Action
0	ϕ	aucune
1	1	lecture
1	0	écriture

ou

`rd`	`wr`	Action
0	0	aucune
1	0	lecture
0	1	écriture
1	1	illégal

Déroulement d'un accès mémoire – Écriture



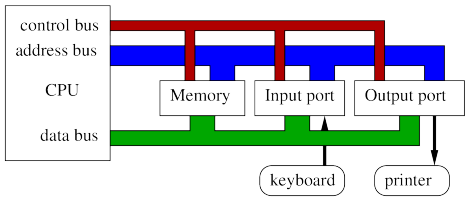
Protocole pour une écriture

1. le CPU place l'adresse `A` sur le bus d'adresse
2. le CPU place la donnée `D` à écrire sur le bus de données
3. le CPU positionne les signaux de contrôle : `Access` $\leftarrow$ 1 et $r/\overline{w} \leftarrow 0$

Résultat : la valeur `D` est mémorisée dans l'octet d'adresse `A`

`*raddr = rdata` `str r0,[raddr]` `Mem[raddr]` $\leftarrow$ `r0`

Déroulement d'un accès mémoire – Lecture



Protocole pour une lecture

1. le CPU place l'adresse `A` sur le bus d'adresse
2. le CPU positionne les signaux de contrôle : `Access` $\leftarrow$ 1 et $r/\overline{w} \leftarrow 1$
3. la mémoire place la donnée lue `Mem[A]` sur le bus de données

Résultat : la valeur `D` est **copiée** depuis `Mem[A]` vers `r0`

`rdata = *raddr` `ldr r0,[raddr]` `r0` $\leftarrow$ `Mem[raddr]`

Pointeur ou adresse ?

Quelle différence entre un pointeur et une adresse ?

↪ aucune !

Les pointeurs "habituels"

```
int* tab;                          // déclaration
tab= (int*) malloc(TAILLE);        // allocation

tab[5] = 18;                       // accès en écriture
x = tab[5];                        // accès en lecture
```

Accès à une adresse

```
*(0x1234FEED) = 14;                // accès en écriture
x = *(0xFEED0000)                  // accès en lecture
```

Certaines adresses ont un rôle particulier...

Comment connecter des périphériques ?

Accès aux périphériques en langage C

- comment transférer des données
  1. vers un périphérique de sortie ?
  2. depuis un périphérique d'entrée ?
- comment décrire ces transferts en **langage C** ?

Seule possibilité : les accès mémoire

- *Memory mapped I/O* – E/S cartographiées en mémoire
- uniques primitives de transfert :
  - lire à une adresse :
    `valeur_lue = *adresse_du_peripherique_entree;`
  - écrire à une adresse :
    `*adresse_du_peripherique_sortie = valeur_ecrite;`

Sémantique des accès aux ports I/O (1/3)

Adresse débanalisée

Registre d'E/S = adresse débanalisée (*à comportement spécial*)
↪ associée à un port d'entrée/sortie (relié à un périphérique).

Une sémantique particulière

- valeur lue ~~= valeur écrite précédemment~~
  = donnée fournie par le périphérique
- lecture d'un port E = consomme une entrée (*en général*)
- écriture sur un port S = émet une nouvelle donnée vers le périphérique (*en général*)

Sémantique des accès aux ports I/O (2/3)

Accès mémoire et registres

Attention, les registres d'E/S **ne sont pas internes** au processeur.

En particulier, on ne peut pas :

- les désigner par leur nom : pas comme `r0`, `pc`, `fp`, ...
- les utiliser comme opérande dans les calculs du processeur :
  ~~`add r0, RNG_DATA, #1`~~
- y accéder directement dans le processeur

Pour y accéder, on passe **forcément** par les bus (données, adresse, contrôle)

- du point de vue du périphérique, c'est donc un **registre** ...
- ...mais du point de vue du périphérique, c'est une adresse mémoire
  ↪ accès via `ldr/str`

On a donc une sémantique différente selon l'adresse accédée.

Sémantique des accès aux ports I/O (3/3)

Conséquence sur la compilation

- il ne faut pas optimiser les accès aux port E/S
  ↪ deux accès consécutifs **doivent** apparaître dans le binaire
- utiliser l'attribut `volatile` dans les accès aux ports E/S

```
volatile uint32_t* mon_entree;
uint32_t premiere_valeur = *mon_entree;
uint32_t deuxieme_valeur = *mon_entree;
```

Sémantique différente du C "habituel" :
`premiere_valeur` et `deuxieme_valeur` peuvent différer !

Macros pour les accès E/S

Rappel : macros et directives de préprocesseur

```
#define MA_MACRO 54
int a = 4 + MA_MACRO
```

`MA_MACRO` remplacé par 54 avant même la compilation
↪ le jour où `MA_MACRO` change, une seule modification du code !

Intérêt de définir des macros pour les accès E/S

- portabilité et évolution des plateformes
- erreurs d'inattention
- facilité de programmation
- **ne pas manipuler directement les adresses des ports !**

Macros pour les accès E/S : exemple d'utilisation

Accès via pointeurs

```
#define INPUT_PORT_PTR ((volatile uint32_t *) 0x10000)
#define OUTPUT_PORT_PTR ((volatile uint32_t *) 0x10004)

uint32_t lu,ecrit;
lu = *INPUT_PORT_PTR;        // lecture port d'entrée
*OUTPUT_PORT_PTR = ecrit;    // écriture port de sortie
```

Accès via variable ordinaire (cas de la libopencm3)

```
#define INPUT_PORT_PTR (* (volatile uint32_t *) 0x10000)
#define OUTPUT_PORT_PTR (* (volatile uint32_t *) 0x10004)

uint32_t lu,ecrit;
lu = INPUT_PORT_PTR;        // lecture port d'entrée
OUTPUT_PORT_PTR = ecrit;    // écriture port de sortie
```


Macros pour les accès E/S : erreurs classiques

- erreur de **typage** :
  ↪ pointeur *vs* variable
  `(volatile uint32_t* x)` *vs* `(*(volatile uint32_t* y))`
- adresse erronée : bien se référer à la **spécification**
- oubli du mot-clef `volatile` :

```
[volatile] int x;
while (x != 0) {
    y++;
}
```

Si `x` `volatile`
↪ test de la valeur `x` à chaque tour de boucle

Si `x` non `volatile`
↪ **boucle infinie dans le binaire !** (sans accès à `x`)

Du port au contrôleur de périphérique

On n'échange pas uniquement des données avec un périphérique...

Configuration et synchronisation pour un périphérique d'entrée

- du processeur vers le périphérique :
  ↪ configuration : choix de l'unité de mesure (°C *vs* °F, ...), fréquence d'échantillonage, ...
  ↪ contrôle : acquisition, *stand-by*, ...
  ↪ synchronisation : indique si le CPU est prêt à recevoir une nouvelle donnée (`rcv_ready`)
- du périphérique vers le processeur
  ↪ retours d'erreur
  ↪ disponibilité d'une nouvelle donnée (`data_ready`)

Similaire (mais inversé) pour un périphérique de sortie.

Interface typique d'un périphérique

Contenu d'une interface typique

1. un port de **donnée** (registre `DATA`)
   ↪ entrée ou sortie selon le périphérique
2. un/des port(s) de **commande** (registre(s) `CONTROL`)
3. un/des port(s) d'**état** (registre(s) `STATUS`)

Nom	Adresse	Fonction
`RNG_DR`	0x5006 0808	registre `DATA`
`RNG_CR`	0x5006 0800	registre `CONTROL`
`RNG_DR`	0x5006 0804	registre `STATUS`

Exemple : générateur de nombre aléatoire (`RNG`)

Sémantique d'un vecteur binaire

Que vaut le vecteur binaire 0x1100 1100 ?
↪ ça dépend de quoi on parle !

Que stocke-t-on dans un vecteur binaire ?

- uniquement la donnée en question
- mais on a besoin d'**information supplémentaire** pour l'interpréter !

Attention à ce que vous manipulez

Adresses *vs* données

Dans ce module, on va manipuler :

- des données :
  valeurs à écrire sur une LED, nombre aléatoire généré par un RNG
- des adresses :
  adresse de la LED orange (sortie), adresse du RNG (entrée)

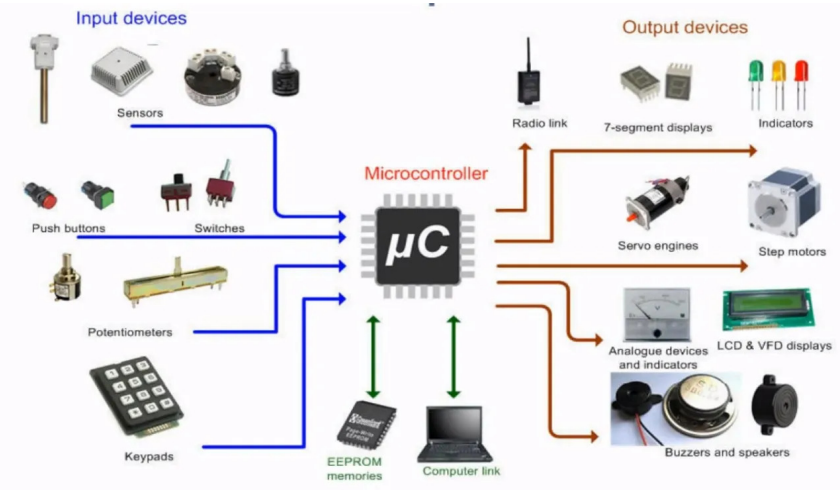
Tout est un vecteur binaire

- représenté et manipulé le plus souvent en **hexadécimal**
  ↪ par exemple : 0x1234 FEED

```
*(volatile uint32_t *)0x40020414 = 0x0180
```

↪ on affecte une **donnée** (0x0180) à une **adresse** (0x40020414)
↪ effet : on allume les LEDs rouge et orange !

Usage typique d'un microcontrôleur



https ://www.electroniclinic.com/microcontroller-8051-avr-and-pic-microcontrollers/

Des microcontrôleurs presque partout

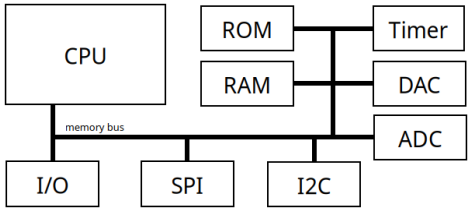


http ://www.vlsifacts.com/wp-content/uploads/2015/10/uC-Applications.png

Travaux Pratiques "bas-niveau" sur carte ARM

Microcontrôleur monopuce

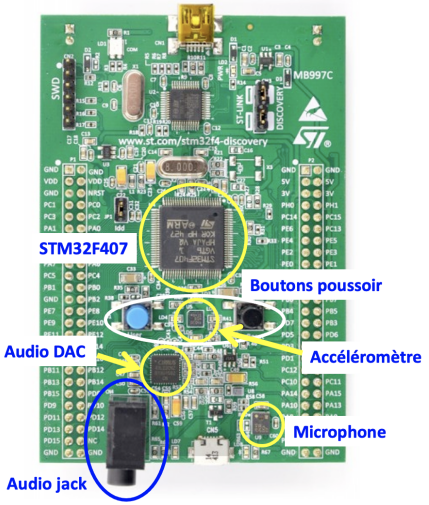
- CPU
- mémoire (ROM+RAM)
- périphériques internes
- entrées/sorties



Rapide présentation carte Discovery F4

- CPU : ARM Cortex M4 STM32F407
- périphériques internes : TIMER, DAC, ADC, ...
- bus série SPI et I2C vers périphériques externes
  ↪ TPs accéléromètre et circuit audio
- ports E/S parallèles vers périphériques externes
  ↪ TPs LEDs et bouton poussoir

La carte Discovery F4



Microcontrôleur STM32F407 : caractéristiques (partielles)

- ARM CorteX M4
- contrôleur d'interruption `plic`
- compteur `systick`
- 192 kio RAM : données, programme
- 1 Mio EEPROM : `firmware`
- ports d'entrée/sortie 16 bits : $\texttt{GPIO}_x$
- bus d'entrée/sortie série : $\texttt{SPI}_x$, $\texttt{I2C}_x$ (+USB)
- générateur de nombres aléatoires : `RNG`
- DAC/ADC : conversion numérique $\leftrightarrow$ analogique

Microcontrôleur STM32F407 : différents périphériques

- 4 LEDs de couleurs (vert, bleu, rouge et orange)
- un bouton poussoir
- un circuit audio (configuration par bus `I2C`)
- un accéléromètre (commande par bus `SPI`)

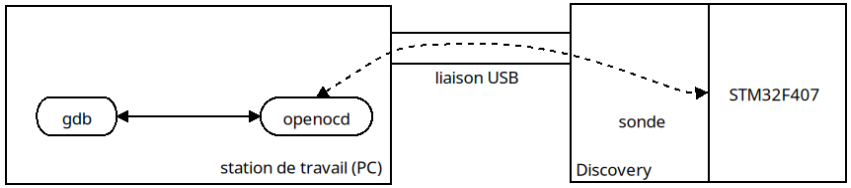
on les manipulera en TP !

Connexion de la carte (1/2)

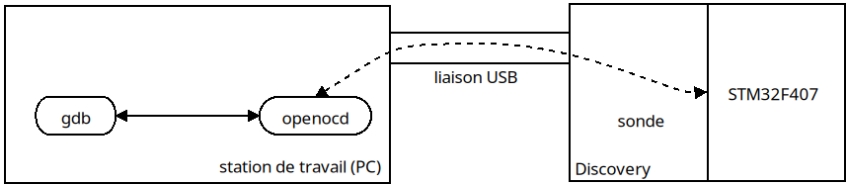
Connexion via USB

La carte intègre une sonde reliée au bus USB, qui permet de :

- alimenter la carte
- lire la ROM et la RAM
- programmer la ROM et écrire dans la RAM
- consulter et modificer les registres du processeur
- contrôler l'exécution : points d'arrêts (*breakpoints*) matériels, pas à pas, *reset*



Connexion de la carte (2/2)



`gdb-multiarch : target remote | openocd`

Attention

`Ctrl-Z` est votre **ennemi** !
↪ suspend et ne termine pas openocd et gdb
`Ctrl-C + quit` pour arrêter proprement gdb

La suite au prochain cours

Contenu du prochain cours

- périphériques disponibles
- configuration des périphériques
- interruptions
- protocoles
- bibliothèque pour interagir avec la carte
- et plus encore...

Exercice

Supposons qu'on a un thermomètre numérique comme périphérique, avec un port d'entrée qui renvoie la température en Kelvin. Ce port d'entrée est défini avec la macro `#define THERM_PORT_PTR ((volatile uint32_t) *) 0x5000`.

Écrire la fonction en C `uint32_t get_temp()` qui récupère la température en Kelvin, calcule sa valeur en degré Celsius, et la renvoie.

Aide

Pour rappel, $0K = -273,15°C$